pollak.thebe.de

The MOD Format

Andreas Pollak

54–68 minutes

2024-05-20

The MOD format was introduced with the first tracker program, the Ultimate Soundtracker, released in 1987. The Ultimate Soundtracker itself did not enjoy a big commercial success. Its lasting impact comes from the fact that its code formed the foundation on which most of the successful tackers on the Amiga were built.

One of the stated objectives of Karsten Obarski, the creator of the Soundtracker, was to help improve the quality of the music in Amiga games by providing developers with a tool to compose scores that require only minimal disk and memory space, do not tax the CPU too heavily during playback and are easy to use in projects. The Soundtracker, as well as most other Amiga trackers that appeared later, came with the assembly source code of a playback routine that could be integrated into programs.

Likely because of the usefulness of the module concept for making demos, various groups soon “developed” their own trackers and released them for free. It is no coincidence that most of the trackers on the Amiga look virtually identical. They can be considered forks of the same codebase, as much of their development took the form of disassembling other people’s programs. One consequence of this is that, even though some of tracker properties and behaviours are rather odd and unexpected, they can still be considered “standard”, as they are consistent across most of the Amiga trackers simply because the implementations were so similar.

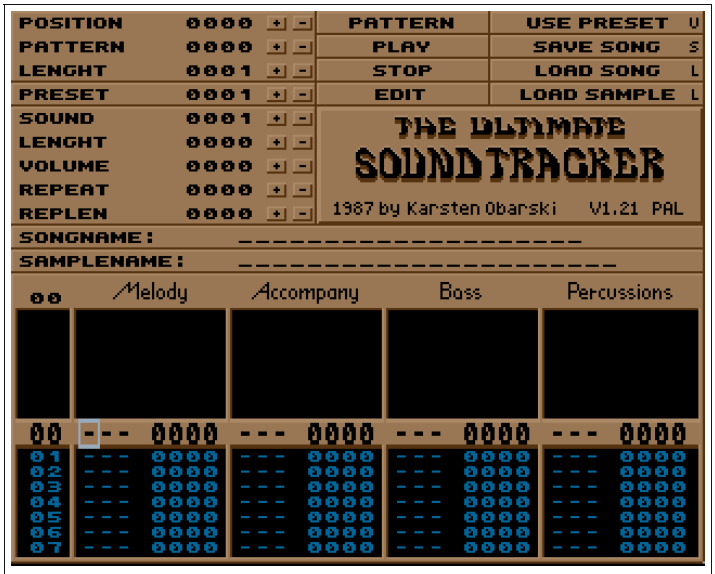
NoiseTracker from 1989 was the first widely used tracker, but it was ProTacker, initially released in the following year, that added lots of improvements and innovations to the concept of the tracker and turned the MOD format into what it is today.

Many of the features and characteristics of MOD files were originally tied to properties of the underlying hardware. Early module players used video timing and processed exactly one tick per frame, which is the reason why the default tempo of modules is still 50 ticks per second. The format was limited to four channels that correspond to the four hardware audio channels on the Amiga, and even the volume and period values used in the MOD format directly match the values that needed

to be written to hardware registers that control audio output. Most of the constraints have been relaxed in one way or another, but lots of traces of the original system the format was developed for remain.

In a way, Obarski's original objective has been achieved thanks to the popularity of the trackers that ripped off the original tracker. Lots of games on the Amiga and PC used module music until at least the mid-1990s. One well-known example is Star Control II, whose developers famously found a very cost-effective way of sourcing high-quality music for their game. They ran a “contest” among module composers, in which the winners would get a little bit of prize money and the honour of having their music included in a video game,

[GDC].



The original tracker: The Ultimate Soundtracker (Amiga, 1987)

File Format

Originating on a Motorola 68k system, the MOD format represents multi-byte words in big-endian format. I will mark such words with BE below as a reminder. Another feature to pay attention to is that **sample positions and lengths** are often, but not always, given in units of **2-byte words**.

MOD File		
size (bytes)	content	description

20	`title`	song title, padded with zeros
15·30 or 31·30	`sampleHeader[]`	15 or 31 Sample Header fields
1	`numSongPos`	number of entries in the pattern table
1	`restartPos`	may contain the restart position of the song; see remarks below
128	`patternTable[]`	list of patterns to be played
0 or 4	`signature`	an identifiable 4-character ASCII signature (31 sample file) or nothing (15 sample file); this field also indicates the number of channels and other aspect of the file format version
$n \cdot c \cdot 256$	`patternData[]`	$256 \cdot c$ bytes of each of the n patterns stored; the number of patterns is derived as the highest pattern index stored in the pattern table plus one; c is the number of channels, which is 4 unless otherwise indicated in the signature field
?	`sampleData[]`	or each sample (15 or 31 of them) 2·`length` bytes as given in the corresponding sample header

Sample Header		
size (bytes)	content	description
20	`sampleTitle`	sample title, padded with zeros; typically these strings are not actually sample descriptions, but form a message when read in order
2 (BE)	`length`	sample length, **in words**
1	`finetune`	bits 0-3 are a signed finetune value
1	`volume`	0..64
2 (BE)	`repeatStart`	sample loop start, **in words**
2 (BE)	`repeatLen`	sample loop length, **in words**

Number of Samples

I will be using the terms sample and instrument interchangeably. MOD files come in two versions, the original

15-sample one introduced by the Ultimate Soundtracker and an extended 31-sample flavour. The 31-sample one can be recognized by having a 4-character ASCII signature at file offset 150+31·30. The original signature indicating a 31-sample file was “M.K.”. Lots of additional signatures were introduced later to identify trackers or non-standard features, most importantly giving a number of channels other than 4. Many are listed in [MMW]. Here’s a summary:

standard 31-sample file:	“M.K.”, “M&K!”
more than 64 patterns:	“M!K!”
giving the number of channels:	“xCHN”, “xxCH”, “xxCN”, “TDZx”, “FLTx”; x or xx is an ASCII decimal number giving the number of channels, e.g. “12CN” or “FLT8”; “CD81”, “OCTA” and “OKTA” indicate 8-channel mods. “FLT8” is a bit of a special case that needs to be handled differently from the other file versions as explained below.

If no signature field is present, try reading the file as a 15-instrument mod.

Any number of channels that's not is a post-ProTracker format extension.

Song Position, Restart Position, Pattern Table

The `patternTable` is the order list of the MOD file. It lists all patterns indices in the order in which they are to be played. Legal values are 0 to 63, unless the `signature` “M!K!” indicates that there may be more than 64 patterns. **Scan all 128 entries** to find the highest referenced pattern. This is the last pattern saved in the file, i.e. in a file with

patterns, this index would be

.

Only the first `numSongPos` entries of the `patternTable` are actually played. Song positions can range from 1 to 128.

`restartPos` may indicate the position in the pattern table to jump to after going though all song positions once.

[MMW] provides some discussion, but the gist of it is that values of $78 and $7F should be ignored, while other values below `numSongPos` are likely valid restart positions. If no restart position is provided, restart from the beginning. This is largely a NoiseTracker feature that's not supported by ProTracker.

pattern data

Each pattern is composed of 64 rows, also called divisions. For each row, there are there are as many cells as there are channels. Every cell is 4 bytes long and contains information on the period, instrument and effect to be played. The four bytes $nm, $lk, $ji, $hg refer to period $mlk, sample $nj, effect $i and effect parameter $hg. (for example, $13, $58, $E2, $AB means period $358, sample $1E, effect 2 and parameter $AB; see the figure below for an illustration) The reason why the sample index is split between two bytes is that when the format was extended from 15 to 31 samples, the extra sample index bit needed to be put somewhere. The upper nibble of the period was unused, as period lengths above 10 bits were invalid anyway.

<table>
<tr><td>byte</td><td colspan="2">0</td><td colspan="2">1</td><td colspan="2">2</td><td colspan="2">3</td></tr>
<tr><td>nibble</td><td>H</td><td>L</td><td>H</td><td>L</td><td>H</td><td>L</td><td>H</td><td>L</td></tr>
<tr><td rowspan="2">data</td><td></td><td colspan="2">period</td><td></td><td rowspan="2">eff</td><td colspan="2" rowspan="2">effParam</td></tr>
<tr><td colspan="4">sample</td></tr>
</table>

Sample numbers run from 1 to 15 or 31. A sample number of 0 in a cell means no new sample is provided, so the old sample information remains unchanged. Similarly, a period value of zero indicates that there is no new note to be played. There is no corresponding null-value for the effect. The convention is to use effect 0 (arpeggio) with a zero parameter, which happens to do nothing of consequence anyway, to indicate "no effect".

Within each pattern, the cell data are organized by row and channel:
[row 0, ch 1],[row 0, ch 2] ... [row 0, ch c],[row 1, ch 1], ...

FLT8 – The Startrekker exception

Startrekker was one of the earlier trackers supporting 8-channel modules through software mixing. Instead of putting additional channel entries into each pattern, it plays two standard 4-channel patterns at the same time, as described in

[STA]. An entry of `pat` in the pattern table means the 8 channels in patterns `pat` and `pat+1` are to be played. The best approach to dealing with this format version may be to divide all pattern table entries by two and to merge pairs of adjacent 4-channel patterns into a single 8-channel pattern upon loading.

Samples

Each sample header contains information on the length of the sample and the start position and length of its loop. These lengths are given in units of 2-byte words, so they need to be multiplied by 2 to get actual byte lengths or positions. The sample will **loop if the repeat length is greater than 1** (not greater than zero), i.e. if the loop is at least 4 bytes long. Some sources (e.g. in [MOD]) state that sample with a loop will play through completely to the end of the sample once before looping. Others state that the loop behaves as expected, i.e. samples will play from the start to the end of the loop, then jump back to the start of the loop. The truth is that ProTracker follows the former behaviour if the loop start is the beginning of the sample (offset 0), and the latter description otherwise (tested on ProTracker 2.1 and 3.1).

The sample data for each sample are stored consecutively after the pattern data as signed bytes. Upon loading, the first two bytes of each sample are overwritten with zeros (ProTracker 1.0 would zero out the first 4 bytes, starting with version 1.1 only 2 bytes).

The sample header also includes information on the default volume of the sample and a finetune value.

```
en msg till jaffaseta
: ole iloinen ja
nauti tasta!!!! sulta
!se! varmaan onnaakin
hihi ruuleaas
```

The sample names form a message that can be read in the MOD player.

Playing the module

In what follows, whenever I describe specific tracker or player behaviour, I refer to how things work on ProTracker. I have experimented with different versions of ProTracker, as well as NoiseTracker and OpenMPT to figure out how instructions and effects are processed.

A player processes patterns in the order in which they are listed in the first `numSongPos` entries of the pattern table. After that, it starts over either at restartPos if provided or at the beginning. Each pattern is processed row by row. The initial speed settings are 6 ticks per row and a tempo of 125, meaning one tick takes .

On the first tick of a row, for each channel, the corresponding cell is evaluated. If a sample is provided, it is set up for the channel; if a period is included, a new note gets triggered; if there is a valid effect in the cell, it is processed as well. For the remaining ticks of the row, ongoing effects continue to be processed.

Most of these behaviours can be modified by effects, as discussed below.

While rows and ticks are processed on a fixed timing schedule, the sample that's active on each channel effectively plays in the background at the current volume and the rate that corresponds to the current period. Non-looping samples just play through once, whereas looping samples continue playing until they are replaced by a different sample.

Channels

The original MOD format supports 4 channels, which correspond to the hardware audio channels available on the Amiga. The Amiga supported stereo sound, with audio channels 1 and 4 permanently mapped to the left and 2 and 3 mapped to the right stereo channel. Due to the limitations of the audio hardware, there was no way of adjusting the panning position of channels in ProTracker. But there are extensions to the MOD format that provide panning effects (effects 8xx and E8x).

If a non-standard number of channels are present in the MOD file, their mapping to the stereo output channels should follow the same pattern as for 4 channel MODs, e.g. LRRL-LRRL for 8 channels.

Processing order

Channels are processed in ascending order, 1 to

. The processing order matters to the extent that certain effects, such as “set tempo” or “pattern break” affect global settings.

When starting a new row, on the first tick, the new instrument is latched first, then the note is triggered, and then most effects are processed. Some effects are dealt with earlier, which will be discussed below.

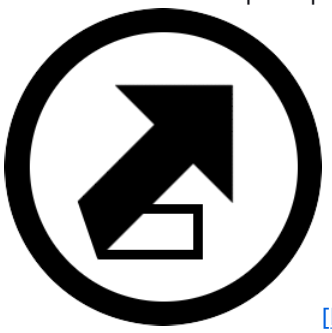
Latching and playing samples

The usual expectation for modules is to either have a valid note and a valid instrument in a cell, with the intended effect that the note will be triggered for that instrument, or to have neither, in which case nothing new happens and any note that may already be playing in the channel remains unaffected. Trackers typically encourage this pattern by automatically inserting an instrument when a new note is entered in a cell. It is still possible to have notes without instruments or vice versa in a cell, and different trackers and module formats deal with such situations differently. Lots of modules depend on what happens is such cases.

I will therefore discuss in some detail how the NoiseTracker/ProTracker family of trackers deals with sample and period information separately instead of just explaining what happens when a note gets triggered. OpenMPT recreates these behaviours faithfully for MOD, thus acknowledging them as the proper standard.

To understand how instruments are handled, it is important to remember that from the perspective of the player, they are not much more than a pointer to the sample data, combined with some length and loop information.

Whenever a new sample is provided in a cell, it is “latched” (see



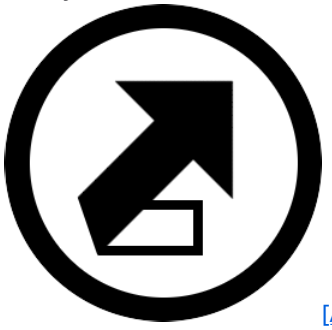
[MMW]), meaning all the relevant information for playing it is copied to a channel-specific data structure for future use. This information includes a pointer to the start of the sample, the default starting index of zero, and the loop start and length, as well as the other two variables defined in the sample header, the finetune value and the default volume. The new volume is applied to the channel immediately, but the other changes may not have an instantaneous effect on what’s playing in the channel unless the sample is retriggered as well.

On the Amiga, the audio hardware (the Paula chip) used DMA

to independently play a whole sample without any further CPU support. When a new sample is latched, the old one keeps on playing until it the new sample is triggered or the old sample reaches the end of its current run. If the previous sample reaches the its end (or the end of its loop), the channel uses the information in the latched sample to decide what to do: either start playing the loop of the newly latched sample if it has one or play nothing. If the previous non-looping sample had already reached its end point and stopped playing, the loop of the newly latched sample will start playing immediately.

In ProTracker, the latched instrument values are initialized such that triggering a note before specifying an instrument in a channel does not play anything. However, there are some modules that do trigger notes before setting a sample, suggesting that there are trackers out there that handle this situation differently.

Samples should be scaled by the current volume and played at a rate that's inversely related to the current period , , where is the relevant system clock (this happened to be the chroma clock, timing colour output on the screen), =3,546,895 Hz on PAL systems and =3,579,545 Hz on NTSC, see



[AHR], pages 140-141. The period found in MOD files is really just a counter value used by the Amiga audio hardware to divide the clock signal. Depending on which clock constant is used, the pitch of samples differs by just under 1% or 0.15 of a semitone. Originally, all trackers were written for PAL systems and the MOD scene was very much concentrated in PAL territory. However, with the move from the Amiga to the PC, new trackers generally adopted the NTSC parameters to determine playback rates.

Triggering Notes

Whenever a non-zero period value is found in a cell, a note gets triggered. This involves two actions: updating the current period and playing the latched sample from its starting index.

The second part is straightforward, the only slight complication to consider is that an effect 9xx in the same row may require an update to the starting index before triggering the sample, as explained below.

The first part is not difficult either. In contrast to other module formats that require the calculation of a period value based on a note index and instrument properties, MOD files directly provide the period corresponding the note that is supposed to be played. This period value may have to be adjusted if the

currently latched instrument has a non-zero finetune value.

Finetune values range from -8 to 7 and are supposed adjust the pitch of a sample by 8ths of a semitone. For example, a note of C2 would normally, for a finetune value of 0, be played at a period of 428. For a finetune value of -5 ($B), the period should be adjusted by a factor of to 444, resulting in a pitch that's about 3.7% or 5/8 of a semitone lower. (Actual finetune period values were tabulated and not always exact, as explained below.)

An effect E5x on the same row may require updating the finetune value before it is used to calculate the period.

On the Amiga trackers, playable notes were limited to a range of three octaves, C1 to B3, corresponding to default period values of 856 to 113. The upper limit of B3 was a result of hardware limitations. In fact, the period 113 was already pushing the limits, being below the minimum of 123 to 124 mandated by the Amiga Hardware Reference Manual to ensure a reliable DMA transfer of audio data (see [AHR], p. 141). For modules made on or for the Amiga, the period values should always be clamped to fall into the allowable range. This is not so much an issue for triggering notes as it is for effects involving frequency slides such as 1xx and 2xx. These constraints can be relaxed for modules that were created post-ProTracker, for example to include octaves 1 and 4 as suggested in [MOD]. Modules that have notes outside of the Amiga period limits are not uncommon. However, some constraints still have to be applied, as periods approaching zero are always problematic.

In ProTracker and other trackers at the time, the periods associated with notes were tabulated (see

[MOD] for such a table). In fact, there were tables for all possible finetune values. When writing just a player rather than a tracker, tabulation of notes is not helpful *per se*, as period values are supplied in the MOD file anyway (finetune-deviations from standard note periods are not, though).

Effects

The Ultimate Soundtracker only supported five effects. The canon of currently used effects was largely introduced by NoiseTracker and ProTracker, and the popularity of ProTracker made it a de-facto standard.

The following table summarizes the available effects. In some cases, a more detailed description or some background information is provided below (column "D" indicates additional general information, column "T" means that there are ProTracker-specific details). The table also indicates whether an effect has “memory” in the sense that an effect parameter of zero means “use the last non-zero parameter for this effect” (an entry "E" in column "M" signifies effect memory). A similar table can be found in [MPT].

The two important note-specific variables that are affected by many effects are the period and the volume. It is useful to think of their current values as having two components, a persistent one that is initially set when a note is triggered or an instrument is latched and that retains its value until it is deliberately changed, and a temporary one that is reset to zero at the beginning of each tick. The effective current volume is then `v_eff=v+v_temp` and the effective current period is `p_eff=p+p_temp`. These effective values determined for each tick are then used for the playback of the sample during the duration of the tick, with the understanding that both the permanent and effective values are always clamped to stay within their allowable bounds. I will use the symbol T for the total number of ticks in a row and `tick=0...T-1` for the current tick. Normally, notes are triggered on the first tick of the row, `tick=0`.

As mentioned, some effects have a “parameter memory”. If used with a non-zero parameter, they copy that parameter value to their memory and then only used the memorized parameter to execute the effect. I will use notation like `eff4_x` to refer to the memory value of an effect parameter. The memory value should be updated when starting to work on a cell before any actual effects are processed. The parameter memory as well as other effect-specific states are separate for each channel unless explicitly noted otherwise.

effect	name	description	D	T	M

0xy	Arpeggio	on every tick, cycle between note, note+x semitones, note+y semitones
		`if (tick%3==1)`
		`p_temp=p*(2^(x/12)-1)`
		`if (tick%3==2)`
		`p_temp=p*(2^(y/12)-1)`
			D	T	-

1xx	Portamento Up	increase pitch by xx on every tick except the first `if (tick>0) p=p-xx`	-	-	-
2xx	Portamento Down	decrease pitch by xx on every tick except the first `if (tick>0) p=p+xx`	-	-	-
3xx	Tone Portamento	set portamento target if note is provided and slide towards the target by xx on every tick except the first; glissando (effect E3x) affects the type of the slide	D	T	E
4xy	Vibrato	perform vibrato with speed x and depth y using the waveform set by effect E4x on every tick except the first	D	T	E
5xy	Volume Slide with Tone Portamento	This executes two effects (Axy and 3xx) at the same time: equivalent to Axy with 300	-	-	-
6xy	Volume Slide with Vibrato	equivalent to Axy with 400	-	-	-
7xx	Tremolo	perform tremolo with speed x and depth y using the waveform set by effect E7x on every tick except the first	D	T	E
8xy	Set Panning	set panning value for the channel to xx (0=left, $FF=right) *this is not a ProTracker effect*	D	-	-

9xx	Set Sample Offset	set the starting offset of the current sample to 256·xx	D	T	E

Axy	Volume Slide	increase the volume by x or decrease it by y on every tick except the first
		`if (tick>0) && (y==0) v=v+x`
		`if (tick>0) && (x==0) v=v+y`
			-	T	-

Bxx	Position Jump	after processing this row, set the song position to xx and continue with row 0 of the corresponding pattern	D	T	-
Cxx	Set Volume	set the current volume to xx `v=min{xx,64}`	-	-	-
Dxy	Pattern Break	after processing this row, continue playing with row 10·x+y of the next song position; **note that xy is decimal**	D	T	-
Fxx	Set Speed	set ticks per row (for xx<$20) or tempo (for xx>=$20) to the indicated parameter value; xx=0 is invalid	D	T	-

NoiseTracker supported effect Exx, where xx=0 would turn on an audio filter, xx=1 would turn it off. With only one bit of the parameter value being used, ProTracker repurposed this this effect by letting the upper nibble of the effect parameter select a sub-effect, while making the lower nibble its effect parameter. This added 15 effects with 4-bit parameters to the MOD format.

effect	name	description	D	T	M
E0x	Set Filter	x=0 turns on a low-pass filter, x=1 turns it off	D	T	-
E1x	Fine Portamento Up	increase pitch by x on the first tick `if (tick==0) p=p-x`	-	-	-
E2x	Fine Portamento Down	decrease pitch by x on the first tick `if (tick==0) p=p+x`	-	-	-
E3x	Glissando Control	enable (x=1) or disable (x=0) glissando for effect 3xx	D	T	-
E4x	Set Vibrato Waveform	choose the waveform to use with effect 4xy	D	T	-
E5x	Set Finetune Value	set the finetune value for the current instrument to x	D	T	-

E6x	Loop Pattern	set the current row as the loop target (x=0) or jump back to the loop target x times from the current row (x>0)	D	T	-
E7x	Set Tremolo Waveform	choose the waveform to use with effect 7xy	D	T	-
E8x	Set Panning	set panning value for the channel to x (0=left, $F=right) *this is not a ProTracker effect*	-	-	-
E9x	Retrigger	retrigger the current instrument every x ticks if x>0; for x=0 do nothing	D	-	-
EAx	Fine Volume Slide Up	increase the volume by x on the first tick `if (tick==0) v=v+x`	-	-	-
EBx	Fine Volume Slide Down	decrease the volume by x on the first tick `if (tick==0) v=v-x`	-	-	-
ECx	Note Cut	set the volume to zero after x ticks	-	-	-
EDx	Note Delay	delay triggering the note by x ticks	D	T	-
EEx	Pattern Delay	increase the number of ticks in this row from T to (1+x)·T	D	-	-
EFx	Funk Repeat/ Invert Loop	think of the most absurd effect you could apply to a sample loop; then apply it	D	T	-

0xy – Arpeggio

This is one of the original Soundtracker commands. It was used to recreate an effect that was popular on systems with simple tone generators that had only one or very few channels, where quickly switching between different tones was a way of making sounds more interesting when playing proper chords was not technically possible. It's not a terribly popular effect, but when it is used, it is immediately recognizable and normally creates a sound that's reminiscent of 1980s console or arcade chiptune music.

If xy=0, this effect does nothing, so 000 can legitimately be considered to mean "no effect".

Tracker-specific:

I'm pleased to report than on ProTracker, this effect works exactly as you'd expect. The bass note of the chord is indeed the tone that's currently playing, even if it is the result of a slide, and the other two are relative to it. For example, if you play a C1, then an arpeggio, then decrease the period by just one and

play the same arpeggio effect again, all three tones will be slightly higher before. Why would I even point that out? After seeing how messy all this is in ScreamTracker (see post on the S3M format), I figured I should make clear that this effect should really work as you thought it should work.

3xx – Tone Portamento

This effect remembers two things: its last non-zero effect parameter `eff3_xx` and a target period `p_target`.

If the effect is in the same cell as a note, the finetune-adjusted period for the current instrument is calculated as usual, but the note is not triggered. Instead, this period becomes the target period `p_target`. As long as the effect is active, the period `p` is adjusted towards `p_target` on every non-zero tick:

```
if (tick>0)
    if (p<p_target) p=min{p+eff3_xx,p_target}
    if (p>p_target) p=max{p-eff3_xx,p_target}
```

It's common to have a 3xx with a note followed by 300 without notes to continue the slide.

This effect (as well as the related 5xy) must be identified by the player before triggering a note.

The effect is modified if glissando is turned on (see effect E3x below).

Tracker-specific:

ProTracker resets the slide target after completing a slide. If you slide partially towards a note, play a different note, then use effect 3xx again without a note, there will be a slide towards the original target period. If you complete a slide towards a note, then play a different note, and use 3xx without a note, nothing happens.

4xy – Vibrato

Vibrato oscillates the frequency of a note.

The state of the vibrato effect has 3 values: current speed `eff4_x` and depth `eff4_y` are part of the effect parameter memory and updated separately (i.e. a command 4xy sets both, 4x0 and 40y only one and 400 sets neither); and the wave table index `eff4_wti` may be initialized to zero if a note is triggered (see E4x for details). Vibrato uses `eff4_wti` on a table set by effect E4x to look up the value of a wave with period 64. Wavetable values should be interpreted as fixed point values in the range between -1 and 1. The default wave is a sine.

The effect adjusts the effective period according to the current wavetable value scaled by twice the depth `eff4_y`, then increases the wave table index by speed `eff4_x`:

```
if (tick>0)
    p_temp=eff4_y*2*eff4_wave[eff4_wti]
```

```
    eff4_wti=(eff4_wti+eff4_x)&63
```

Tracker-specific:

On ProTracker for the rectangular waveform, 4xy adjust the period by –(2y-1) and 2y-1, likely because the wavetable is scaled to be strictly between -1 and 1, such as

.

Vibrato depth was originally different in ProTracker 1.0.

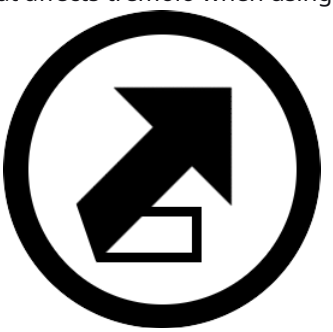
7xy – Tremolo

This effect works very much like vibrato, except that it affects the volume rather than the pitch of a note and is scaled twice as much:

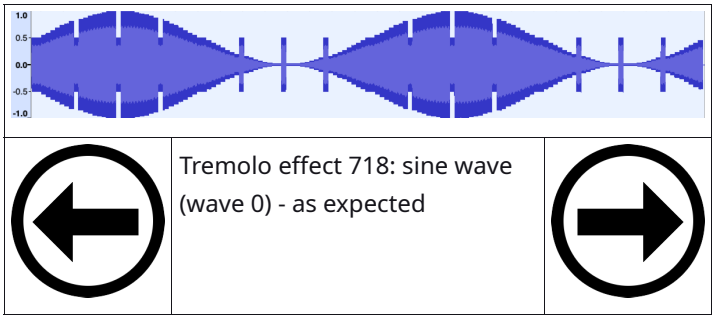
```
if (tick>0)
    v_temp=eff7_y*4*eff7_wave[eff7_wti]
    eff7_wti=(eff7_wti+eff7_x)&63;
```

Tracker-specific:

There is a bug in ProTracker that affects tremolo when using



the ramp-down waveform, see [TTF]. The ramp down, which would be expected to look like a ramp up, really, in the case of tremolo (see effect E7x below), is somewhat messed up. It seems that it's calculated on the fly, but ProTracker is looking at some of the wrong bits when doing so - reportedly bits tied to the vibrato state. The pictures below show some ProTracker 3.1 audio output to illustrate the issue.



These plots show the recorded audio output for tremolo at speed 1. In all cases, a high-pitch tone plays for 16 rows at 9 ticks per row (for a total of 128 non-first ticks, and thus two full tremolo cycles) at a volume of $20. Tremolo depth is set to 8, so that the effective volume varies from 0 to its maximum of $40. The periodic blips are the first ticks of each row, on which no tremolo is applied.

The buggy waveform value (as a fraction, with

) can be calculated like this:

```
wave1=(((eff4_wti<32)?eff7_wti:-eff7_wti)&31)*((eff7_wti<32)?1:-1)/32
```

Here, `eff7_wti` and `eff4_wti` are the wave table indices for tremolo and vibrato, respectively, with values ranging from 0 to 63. Everything would be good if `eff7_wti` had been used consistently in the formula instead of `eff4_wti`.

▸ Note

Why? Shouldn't it be really easy to calculate the value correctly? Here's a hypothesis:

First we calculate the absolute value of the waveform function:

```
wave1_abs=((eff7_wti<32)?eff7_wti:-eff7_wti)&31
```

This is useful, because now we can multiply it by the effect parameter `eff7_y` and do some scaling using *unsigned* multiplication and shifts.

```
v_temp=(wave1_abs*eff7_y)>>3
```

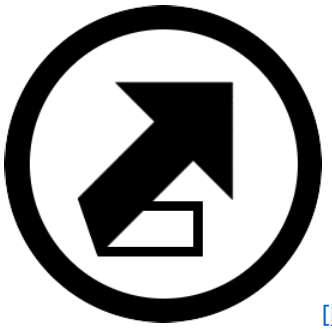
Now we just flip the sign.

```
if (eff7_wti>=32) v_temp=-v_temp
```

All this would be largely copy and paste from the corresponding vibrato processing, which is why in the first step, `eff4_wti` was incorrectly used.

8xx – Panning

This effect, like the similar E8x, is a post-ProTracker extension that allows a module to set the panning position of a channel. On the Amiga, the panning positions of the hardware channels were fixed. This effect required either software mixing or hardware that supports panning. The scaling of xx between 0 and $FF documented here is consistent with the implementations in FastTracker and OpenMPT, but there may be trackers that use this effect differently. Note that

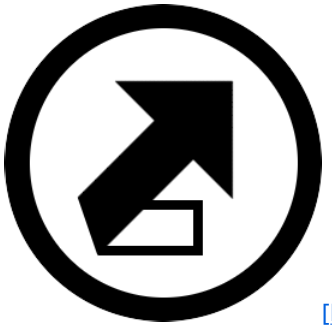


[MOD] documents a different scaling of this parameter, which is known to have been in use. OpenMPT attempts to detect such uses of this effect and handle them correctly.

Despite being an extension to the ProTracker canon of effects, it is an important command. First, quite a few modules use it. Second, panning is a feature that became relevant mostly on the PC, where the MOD format continued to be popular after ProTracker ceased to be the reference implementation for MOD effects.

9xx – Set Sample Offset

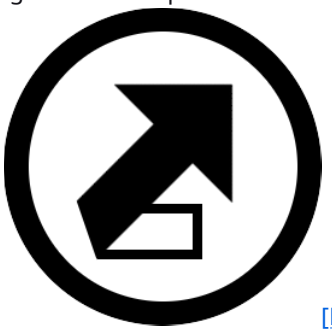
Starts the current sample at index 256·`eff9_xx` rather than 0.



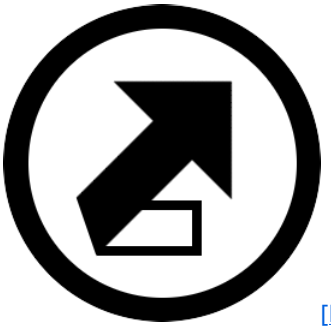
[MOD] states that the offset xx is in words, but ProTracker 1.0, ProTracker 2.1, FastTracker and OpenMPT agree that it's in bytes (or samples). It is possible that some other early tracker used a word offset. This effect is entirely straightforward if a new sample is triggered in the same cell.

Tracker-specific:

Things get more complicated if no sample is provided with the



effect. [MPT] states that nothing



should happen; [MOD] wants us to retrigger any still playing sample at the proper offset. Here is what ProTracker actually does:

If a note is triggered on the current row (non-zero period in the cell), ProTracker checks for a 9xx effect and, if it finds one, adds the proper offset of 256·`eff9_xx` to the starting index (or pointer) of the latched sample. This is done after latching a new sample but *before triggering a note*, ensuring that any new note gets played from the right starting point.

After triggering, when the current effects are processed as usual, this effect again adds 256·`eff9_xx` to the latched starting index, this time to make sure that the starting index is updated for the next use of the latched sample.

This results in the following slightly unusual behaviour: If a note is triggered with effect 9xx and an instrument present, it will play the sample from position 256·xx as expected. If a new note

is then triggered again without an instrument, it will play from 2·256·xx.

Because the parameter gets added to the starting offset, the effect of multiple 9xx commands is cumulative. Providing an instrument always resets the starting point by latching a new sample.

Axy – Volume Slide

Tracker-specific:

For the illegal parameter combination x!=0 and y!=0, ProTracker increases the volume by x.

Bxx – Position Jump

After this row, continue at the first row of the pattern associated with the specified song position xx. This effect interacts and is commonly used with Dxx. See the description of Dxx for details.

Tracker-specific:

If xx is a value greater or equal than numSongPos, ProTracker jumps to song position 0. This includes the case of invalid song positions greater than 127.

Dxy – Pattern Break

This effect ends processing the pattern at the current song position after this row. It continues on row xy of the pattern at the next song position. When originally introduced in NoiseTracker, this effect had no parameters, it just ended the current pattern prematurely, hence the effect name.

There are two things to be careful about: First, the effect parameter xy is specified as a **decimal number**, that means that the target row is calculated as 10·x+y rather than 16·x+y. The reason is probably that row numbers were shown as decimals in trackers, so this practice avoids mistakes, confusion and mental math.

Second, it is a common idiom to combine this effect with Bxx to jump to a particular row at a particular song position. For this to work as expected, the control flow in modules must be handled in a way that consistent with how ProTracker did it. One way of doing this is to use a pattern break flag that indicates if a new song position should be loaded:

Dxy sets `pattern_break=true` and `row_next=x*10+y`

Bxx sets `pattern_break=true` and `row_next=0, songPos_next=xx`

The routine that updates the current row and song pos after a row is completed then does something like this:

```
row=row_next
```

```
if (row>64)
    row=0
    pattern_break=true
row_next=row+1
if (pattern_break)
    songPos=songPos_next
    check if songPos valid, restart song if
necessary, etc.
    songPos_next=songPos+1
    pattern_break=false
```

With this, combining the Bxx with Dxx makes jumps to arbitrary point possible if the effects appear in this order on the same row, i.e. if Bxx is entered in a lower channel than Dyx. Otherwise, if the rightmost jump command on the row is a Bxx, it will fully determine the jump destination. This is exactly the behaviour observed on ProTracker and many other trackers.

Tracker-specific:

If an illegal row above 63 is entered in ProTracker, the command jumps to row 0 on the next pattern, as is implied by the above description. An alternative plausible behaviour would be to ignore Dxy commands with illegal row numbers.

Fxx – Set Speed

This effect was first introduced to change the number of ticks per row at a time when tempo was still tied to the screen refresh rate and trackers invariably processed 50 ticks per second. With the adoption of more flexible timing in ProTracker 2, it was changed to set the tempo for parameters of xx>=$20, meaning that the duration of a tick is set to

.

Even before the effect was extended to allow setting tempo, NoiseTracker already limited the allowable values for ticks per row to be less than $20. [MMW] therefore suggests always interpreting xx as ticks per row for old (pre-NoiseTracker) 15-sample modules, which seems like reasonable advice.

There is some inconsistency with regards to how a value of xx=$20 is treated, with some trackers interpreting it as ticks per row. ProTracker and current versions of OpenMPT take xx=$20 to be a tempo setting, which is consistent with the ProTracker

documentation ([PTM]).

The ProTracker docs also state that a value of xx=0 stops playback. Some trackers implement the effect this way, while others, including OpenMPT, ignore an effect F00. The MOD format does not have a direct way of indicating the end of a song, so some modules use F00 to show that playback should stop.

Tracker-specific:

For an effect parameter of 0, ProTracker stops processing the module after the first tick of the row containing F00. However, any active samples keep on playing, and in fact keep playing forever if they have a loop.

This effect is processed on the first tick. However, tempo changes (x>=$20) do not yet affect the length of the initial tick of the row. Presumably, the interrupt timing for handling the next tick is set up before processing effects.

E0x – Set Filter

The Amiga had a hardware audio filter that could be applied to both stereo channels. It was a second order low-pass Butterworth filter with cut-off frequency of reportedly 3,275 Hz. The filter state is a system-wide hardware setting. The filter was enabled during the boot sequence but turned off upon program start by NoiseTracker and ProTracker. The filter dampens higher frequencies quite a bit.

This effect turns this filter on or off. Notice that counterintuitively, **x=0 enables the filter**. The recommendation, starting with the NoiseTracker manual, has always been to keep the filter turned off. OpenMPT suggests to leave this command alone, unless your module may be played on an Amiga, in which case you may use E01 to ensure the filter is off.

None of the major trackers on the PC in the 90s implemented this effect, as there was no hardware support. Thus, MODs created back then on a PC would likely ignore this effect.

Overall, this is really not an important effect. Most MODs do not use it, some turn the filter off, very few turn the filter on. And if 1990s PC trackers are the reference point, not implementing this effect may be the most authentic choice.

At the same time, if audio output is being filtered anyway, this effect is extremely simple to implement.

Tracker-specific:

ProTracker turns the filter on for x=0, all other values turn it off.

E3x – Glissando Control

Normally, the slide-to-note effect 3xx adjusts frequencies in a continuous way for as long as it's active. Glissando modifies the slide so that it is done in steps of semitones, approximating a slide effect that's more akin to what a guitar would sound like rather than a slide whistle.

This effect turns glissando on (x=1) or off (x=0). Even if turned on, glissando only applies while effect 3xx (or 5xx) is active. By default, glissando is off.

This effect has not been supported as consistently as one might expect. ImpulseTracker does not have it, OpenMPT describes its implementation as "quirky". It is generally considered to be an unimportant, rarely used and little liked effect.

Despite all this, it's not a difficult effect to implement. All it takes is to "round" the current playback frequency to the nearest semitone if glissando and effect 3xx are both active. In ProTracker and other trackers of that time, this was clearly accomplished by using period tables. Today, it can be done in a much simpler way. Just round the relative deviation from any reference note appropriately when calculating the data rates used for playing samples. When tone portamento is active, such a reference semitone is always available in the form of the target period.

```
effPeriod=period
if (eff3xx_active && glissando_active)
    rel_period=log_2(effPeriod/target_period)*12

effPeriod=target_period*2^(round(rel_period)/12)
sample_data_rate=Phi/effPeriod
```

Tracker-specific:

In ProTracker, any parameter value of x>0 will turn glissando on.

E4x/E7x – Set Vibrato/Tremolo Waveform

Vibrato and tremolo both modify a playback parameter in a time-dependent way. By default, they use a sine wave to modulate audio output. The values of this sine wave are provided in a table with 64 entries that have values in the range from -1 to 1. Originally, these values would have been encoded as fixed-point values, but today we can equivalently think of them as floating-point numbers.

The lowest two bits of the effect parameter selects one of four possible waveforms:

0:	sine wave:	`wave0[i]=sin(2*pi*i/64)`

1:	ramp down:	`wave1[i]=((i+32)%64)/32-1`
2:	square wave:	`wave2[i]=i<32?1:-1`
3:	random:	`wave3[i]=?`

Note that, despite being advertised as “ramp down” in the documentation, wave 1 is very much a ramp up in the numerical sense (after triggering, both wave 0 and wave 1 initially increase in value). "Down" probably refers to the direction of the pitch change when the wave is used for vibrato.

Different sources have different opinions on what “random” means. [MOD] claims it means “pick one of waveforms 0-2 randomly”. ImpulseTracker and others implement it as what appears to be a sequence of uniformly distributed, i.i.d. random numbers – basically noise. The OpenMPT docs state that ProTracker does not implement the random waveform and recommends avoiding its use in modules.

Bit 2 of the effect parameter determines whether a wave is retriggered with a new note. If this it is zero, set the wave table indices to zero whenever a new note is triggered. If it’s one, don’t do this. The default is to retrigger.

Bit 3 of the effect parameter does nothing.

Tracker-specific:

The ProTracker docs insist that waveform 3 is random, but ProTracker would only ever give me the square waveform for x=3. It also appears that at least the square waveform is scaled such that its values are strictly between -1 and 1, see the remarks on effect 4xx.

E5x – Set Finetune Value

This effect sets the finetune value for the current note, overriding the one provided in the sample header for the instrument. This effect makes most sense when paired with a note. In OpenMPT, that is the only scenario in which it works.

Tracker-specific:

In ProTracker, this effect is handled similar to 9xx, in the sense that it affects both the current note and the latched instrument. It sets the latched finetune value, overwriting any default value previously copied there from the instrument header. Presumably, the effect is executed up to two times: Once just

before triggering a note or calculating the target period for a slide if a non-zero period was present in the cell. And then later on the first tick, when regular effects are processed. This behaviour does not lead to observable quirks as in the case of the 9xx command, but it ensures that both the current note (if present) and future uses of the same latched instruments are affected.

E6x – Loop Pattern

If the Bxx/Dxy commands are the equivalents of a “goto”, then this effect corresponds to a for-loop. The idea is simple: E60 marks the current row as a loop target, whereas E6x for x>0 means: jump back to the loop target row and loop a total of x times.

How is this implemented in practice? First of all, there’s no jumping between patterns. The loop target is really just a row number. Second, there is a loop counter that ensures that ensures the loop gets processed the right number of times. This is how far a general description of this effect can go. Trackers differ widely in their implementations and how they handle unusual scenarios. I will describe the behaviour of ProTracker here.

Tracker-specific:

In ProTracker, both the jump target and the loop counter are global variables, meaning there is no way of nesting loops across channels. The following pseudo-code summarizes how things work. It is based on experiments with the tracker, not reading the source code:

```
if (x==0) loopTargetRow=row
else
    if (globalLoopCounter<=0) globalLoopCounter=x
    else globalLoopCounter-=1
    if (globalLoopCounter>0)
row_next=loopTargetRow
```

The loop counter and target row are initialized to zero, but then not updated further except by the E6x effect. In particular, the target row is not reset to zero when moving to a new pattern.

This implementation makes it possible to do some unintended things: by setting a row in one pattern, then using a E6x jump early in a later pattern, it is possible to jump forward rather than back in a pattern. Having a E60 followed by two E6x commands in the same pattern will create an infinite loop.

Later trackers handle this command differently. Some have channel specific-loop states, allowing nested loops across channels. Most seem to reset the jump target in E6x commands to prevent infinite loops. Such implementations will be discussed later in the context of the other module formats.

E9x – Retrigger

This effect retriggers the sample, which really just means starting the latched sample at its latched starting position (which may have been modified by 9xx) without changing the current period of the channel.

For x>0, the note gets retriggered on the first tick (even if no period was supplied in the cell so that normal triggering would not happen) and then every x ticks (i.e. on every tick if x=0) on the current row.

For x=0, the effect does nothing, i.e. it does not affect the normal processing of whatever else is in the current cell.

Other module formats have a persistent counter associated with this effect which allows it to operate across rows. The MOD format does not.

EDx – Note Delay

This effect delays triggering the current note by x ticks. If x is greater or equal than the number of ticks, the note is not triggered at all.

In the relatively simple MOD format, there's lots of different (and mostly equivalent) ways of making this happen. However, from looking at more complex formats such as XM or IT, it is clear that this effect actually ends up meaning "delay the processing of all new information in this cell by x ticks". One way of thinking about this is that this effect changes the channel-specific tick index to `channelTick=globalTick-x`. No current effects are processed on the "negative" channel ticks -x to -1, then the new note is triggered and things start happening at `channelTick` 0, when the other channels are already on tick x.

Tracker-specific:

On ProTracker, for a parameter value of x=0, there is no delay at all, so the effect does nothing, really. Also, in blatant violation of the general interpretation of this effect I just gave, ProTracker swaps out the sample immediately while only delaying the triggering.
If x is greater or equal to the number of ticks, the new note is not supposed to be triggered at all. However, the new period is still applied at the beginning of the next row if there is no new note there. The instrument is not triggered, though. So if EDx is applied in this manner, any still playing note will just change pitch at the beginning of the next row. Even if the note has stopped playing, the new period is set and will be the one used for future retriggers.

EEx – Pattern Delay

The current row is made (x+1) times as long as normal by getting an additional x·T ticks added to it. This means that notes are still only triggered once, but the row is longer in terms of "regular" ticks t>0, on which effects are processed as

usual.

This is done by setting (rather than adding to) a global variable, meaning that in the case of multiple EEx commands on the same row, only the one in the highest channel matters.

EFx – Funk Repeat/Invert Loop

Some ideas are better than others, but only the worst ones are considered for inclusion as an EFx effect.

This family of effects was introduced by ProTracker. The EFx command generally controls an effect that operates in the background. The parameter x sets the "speed" of the effect by selecting a value from a table. This value is then used on every tick to increment a counter. When this counter overflows, it is reset to zero and something bad happens. A parameter value of x=0 selects an increment of zero and thus effectively turns the effect off.

ProTracker originally implemented an effect called "Funk Repeat:"

Whenever the funk effect is triggered on a tick, the loop of the current sample is moved forward by the loop length. Once it moves past the end of the sample, it is reset to its original position. It's not entirely clear what this is supposed to achieve. I have not dared to try out how this sounds in practice, so I will quote from the effect description in the on-disk manual that accompanied ProTracker 1.0C: "Sounds like shit, really, but who cares?"

This effect was soon replaced by a reworked version, which was called "Invert Loop", but, confusingly, in some documents as well as ProTracker sources is still referred to as Funk Repeat. The new effect changes what happens when the counter overflows and an event is triggered. Now the effect increases an index and then inverts the sample value at this index inside the loop of the current sample:

```
currSampleData[loopStart+invertIndex%loopLength]^=$FF
```

What does this achieve? Bitwise inversion of the sample's 2's complement value is approximately the same as multiplying it by -1. For a longer loop, this will not change the sound much, except for the introduction of clicking noises between the inverted and uninverted parts. For short loops, it will likely superimpose an audible rectangular wave that is strongly modulated at a rate related to the effect speed. Why would we want that? I have no idea, but [MMW] asserts that a few modules need this effect to enhance

the epicness of their sound.

A clear issue with this effect is that it irreparably modifies actual sample data in the middle of playback. When using a tracker, do we expect it to keep an unmodified copy of the samples or is it ok that samples get mangled more and more every time a module is played? Quoting from the ProTracker 2.1A manual: “This effect will trash the sample, and will probably be removed in the next version.” This promise was not kept.

The good news is that later trackers consistently refuse to implement the EFx effect. My suggestion: follow their example.

Specific details, including the relevant increment table, are explained in [MMW]. A significantly deeper and, from the perspective of implementer, more helpful discussion can be found in [TST].

Tracker-specific:

ProTracker introduced Funk Repeat in version 1.0 and replaced it with Invert Loop in version 1.1.

Finally...

Tabulation

Trackers for the MOD format use tables to map notes to periods. ProTracker tabulated the 36 supported notes across octaves for each of the possible 8 finetune values.

For the purpose of writing a player, this is not a big concern, as the MOD file already supplies the correct period for each note to be played. The only two situations where we would be required to look up a tabulated value is when applying finetune to a period and when using glissando. Above, I explained how the required adjustments can be calculated. The result of these calculations is going to be very close to ProTracker’s table values, but may still be off by a small fraction of a percent. It’s unlikely that this makes an audible difference.

When writing a tracker, using the “official” period values for notes is important, because these values get written to the

MOD file, and non-standard periods may confuse other trackers or players. These period values are listed in [MOD], for example. Trying to calculate these values based on a reference note will not work; some periods will always be off by +/-1. Still, it's sufficient to tabulate one octave (the lowest one to be used), and then calculate the periods for higher octaves by bit-shifting.

For ultimate playback faithfulness to ProTracker, the complete table including periods for non-zero finetune values can be found in [PTT], along with some theories and discussion of why it might be such an inconsistent mess.

Filtering

The Amiga had a built-in 2nd-order Butterworth low-pass filter with a cut-off frequency of about 3.3 kHz (a value of 3,275 Hz is sometimes reported, but the exact specification may well have differed across hardware revisions). This filter could be turned off. On the popular Amiga 500, the status of the filter was indicated by adjusting the brightness of the power LED. It is therefore often referred to as the LED filter, power light filter or similar.

When that filter was off, the Amiga 1000 and 500 (i.e. the pre-1992 models) still filtered audio output with a 1st-order low-pass filter at about 4.9 kHz. See [AMT] for the Amiga filter values allegedly used by MikyTracker

and [AFS] for audio recordings on different Amiga models.

On the PC at the time, SoundBlaster cards filtered audio output quite heavily, with the default being comparable to the Amiga's Butterworth filter (12 dB per octave, and cut-offs at about 3.8 kHz and 3.2 kHz on the SoundBlaster and SoundBlaster Pro, respectively).

In 1992, non-optional filters disappeared, both on the new Amiga models and the new SoundBlaster 16 on the PC. The optional Butterworth filter continued to exist on the next generation of Amigas.

So, what's the right filter for MOD files?

One thing to consider is that the maximum sample rate the format supports in its original form is about 30 kHz for a theoretical maximum tone frequency of 15 kHz. Generally, the inherent restrictions on sample rates in MOD files make it difficult to generate high frequencies in the audio output with reasonable quality. This constraint was relaxed by later module formats that allow higher sampling rates and a wider range of usable octaves. Moreover, the Amiga hardware allowed for variable sample playback rates that could differ across channels, whereas on the PC, having to choose a fixed rate for the whole module was the norm. This means that aliasing was likely much more of a problem on the PC, in particular considering that earlier PCM cards did not support sample rates anywhere near the 44+ kHz that would later be considered the minimum standard. For these reasons, any frequencies above maybe 10-15 kHz could be considered unwanted and would have been largely filtered out anyway, at least on pre-1992 hardware.

For modules that respect the original Amiga period range, the Amiga filter configuration may be a pretty good choice, considering the characteristics of both the format and the hardware at the time of its development. For the ones that do not, it may be hard to determine what's authentic, but I had the impression that applying a filter generally improves the perceived audio quality.

References:

[GDC]	Here's an interview with the creators of the Star Control II game. From approximately minutes 12:00 to 16:00, they talk about game audio. (But the whole video is worth watching, really.)

	https://www.gdcvault.com/play/1021863/Classic-Game-Postmortem-Star
[MOD]	This is probably the most "original" documentation of the format (short of looking at the source code): https://www.aes.id.au/modformat.html
[MMW]	A very detailed description of the file format and effects, but not always as accurate and precise as one would hope: https://wiki.multimedia.cx/index.php/Protracker_Module
[STA]	A small collection of pieces of information on the Startrekker module format: http://www.textfiles.com/programming/FORMATS/starfmt.pro
[AHR]	The Amiga Hardware Reference Manual (AHRM), 3rd edition. Conveniently opened at the beginning of the audio chapter, which explains how to play samples.

	https://archive.org/details/amiga-hardware-reference-manual-3rd-edition/page/133/mode/2up
[MPT]	The OpenMPT manual contains some detailed information on MOD effects. The focus is naturally on the OpenMPT implementation: https://wiki.openmpt.org/Manual:_Effect_Reference#MOD_Effect_Commands
[PTM]	This is "read.me" file that came with ProTracker 3.1, one of the later versions of the program. It includes a description of the available effects. /b/Modules/docs/read.me.txt If this is not enough, the readme files of *virtually all* ProTracker, NoiseTracker and Soundtracker versions are available here: https://github.com/cmatsuoka/tracker-history/tree/master/reference/amiga
[TTF]	This document lists a lot of details about playing MODs in a compatible way. The focus in on FastTracker II behaviour, but, among other things, the tremolo-ramp-down bug is documented here:

	https://weaselaudiolib.sourceforge.net/TakeTracker-FastTracker-notes-and-format.html#e44_7xx_-_tremolosawtoothwaveformbugmod
[TST]	A forum discussion that starts out on tracker testing, but quickly turns to the MOD EFx effects. The discussion provides some insights into how the effected worked in ProTracker and has some code samples. https://www.un4seen.com/forum/?topic=7554.0
[PTT]	This is part of the OpenMPT source. The interesting thing here is the comments at the beginning of the file, containing the ProTracker finetune table and a forensic analysis of how one may have come up with these numbers. https://resources.openmpt.org/documents/PTGenerator.c
[AFS]	Sample recordings from different Amiga models to test audio filters: https://www.paula8364.com/post.php?id=000209032015221937

[AMT]	A brief forum discussion about Amiga filters. It shows the parameters allegedly used by MilkyTracker. https://forum.renoise.com/t/disabling-amiga-internal-filter/46412/2
Here are some links that are not referenced above, but may be of interest.	
[UST]	A brief history of the Ultimate Soundtracker. http://www.textfiles.com/artscene/music/information/karstenobarski.html
[KOB]	And interview with Karsten Obarski, who created the first tracker. https://amp.dascene.net/detail.php?view=3982&detail=interview
[LCL]	A collection of spec files of the major module formats. The MOD document is very comprehensive and includes finetune tables. https://github.com/lclevy/unmo3/tree/master/spec
[CWI]	Collected bits and pieces of information about the MOD format from the early 1990s. Among other things, it includes a documentation of the decompression algorithm

	that can be used to “decrunch” MODs that were packed on the Amiga. https://www.eblong.com/zarf/blorb/mod-spec.txt
[OMC]	The OpenMPT compatibility settings for MOD. They list a number of known glitches. https://wiki.openmpt.org/Manual:_Compatible_Playback#MOD_compatibility_settings